

Resumen de Estudio

Arquitectura de Software

Juan Ricardo Giadach · UDP 2026

Este resumen cubre los 5 módulos del curso: Introducción, Requerimientos No Funcionales y Arquitecturas Genéricas (estructuración + modelos de control), con definiciones clave, ventajas/desventajas y ejemplos de ejercicios.

MÓDULO 1 · Introducción a la Arquitectura de Software

¿Qué es la Arquitectura de Software?

Definiciones clave de autores reconocidos:

G. Booch: Son las decisiones de diseño significativas que dan forma a un sistema, donde 'significativo' se mide por el costo del cambio.

T. Gilb: Es una hipótesis que necesita ser probada mediante implementación y mediciones.

IEEE: Organización fundamental de un sistema: sus componentes, sus relaciones entre sí y con el entorno, y los principios que gobiernan su diseño y evolución.

L. Bass: La estructura (o estructuras) del sistema, que comprende elementos de software, sus propiedades visibles externamente y las relaciones entre ellos.

D. Garlan: Va más allá de los algoritmos y estructuras de datos; aborda la organización global, protocolos de comunicación, distribución física, escalabilidad y selección de alternativas de diseño.

Funciones de la Arquitectura

- Define la estructura (componentes del sistema)
- Especifica la comunicación entre componentes
- Plantea los requerimientos no funcionales: restricciones técnicas, de negocio y atributos de calidad
- Es una abstracción del sistema
- Tiene diversas vistas: lógica, proceso, física, desarrollo

El Arquitecto de Software

- Administración del riesgo técnico
- Conocimiento profundo de la tecnología disponible
- Excelentes habilidades de diseño
- Interfaz entre el cliente y el equipo técnico
- Promoción de buenas prácticas de desarrollo
- Puente de comunicación entre equipos de desarrollo

Contenidos del Curso

1. Introducción a la arquitectura de software
2. Propiedades no funcionales
3. Arquitecturas de Software Genéricas
4. Patrones de Arquitectura
5. Pruebas, medición y validación según atributos de calidad

MÓDULO 2 · Requerimientos No Funcionales (Atributos de Calidad)

Los requerimientos no funcionales definen cómo se comporta el sistema, no qué hace. Son determinantes para la arquitectura.

Atributo	Descripción	Estrategias Arquitectónicas
Performance (Rendimiento)	Respuesta eficiente. Métricas: throughput, tiempo de respuesta, plazos.	• Pocos componentes • Operaciones críticas concentradas • Minimizar comunicación entre componentes • Bajo uso de red e I/O
Escalabilidad	Aceptar mayor carga sin degradación. Métricas: tps, conexiones simultáneas, volumen de datos.	• Componentes auto-contenidos • Bajo acoplamiento • Crecimiento horizontal (scale-out)
Mantenibilidad	Soportar nuevos requerimientos funcionales con bajo costo.	• Componentes auto-contenidos • Especificación detallada • Componentes reemplazables • Evitar datos compartidos
Seguridad	Proteger acceso y datos.	Nivel usuario: autenticación, autorización, perfilamiento. Nivel datos: encriptación, integridad, no repudio
Confiabilidad	Sistema siempre activo. Regla de los cinco nueves (99.999% disponibilidad).	• Componentes redundantes • Fácil migración • Bajo acoplamiento • Control distribuido • Activo-activo o activo-pasivo
Integrabilidad	Integración con otros sistemas y disponibilización de APIs.	• Interoperación estándar • APIs bien definidas (REST, SOAP)
Portabilidad	Independencia de plataforma (parcial o total).	• Contenedores (Docker) • Máquinas virtuales
Verificabilidad	Capacidad de autotesteo en tiempo real.	• Transacciones fantasma • Procesos de verificación operativa
Soportabilidad	Manejo de errores, diagnóstico y corrección de incidencias.	• Diseño modular • CI/CD • Bitácora de procesos • Gestión de logs • Documentación actualizada

Citas importantes del módulo:

"The only way to go fast, is to go well." — R. Martin

"If you think good architecture is expensive, try bad architecture." — B. Foote & J. Yoder

"Todo beneficio obtenido tiene un costo asociado." — JRG

MÓDULO 3 · Arquitecturas de Software Genéricas: Estructuración

Fases del Diseño de Software

Fase 0 – Analizar el contexto: Requerimientos funcionales, no funcionales, ambiente operacional y restricciones.

Fase 1 – Estructuración: Identificar componentes y sus relaciones (diagrama de bloques, flujo de datos, interfaces).

Fase 2 – Modelo de control: Definir el comportamiento de los componentes y cómo se controla el flujo.

Fase 3 – Descomposición modular: Diseño detallado de cada componente.

1. Arquitectura Cliente / Servidor

Modelo parcialmente distribuido compuesto por servidores (ofrecen servicios), clientes (consumen servicios) y redes de comunicación.

Ventajas	Desventajas
Procesamiento distribuido Datos distribuidos Escalable	Datos no compartidos entre servidores Administración de datos en cada servidor Performance deteriorada Sin registro centralizado de servicios

Ejemplo – Calculadora:

Una empresa quiere ofrecer una calculadora con las 4 operaciones básicas a sus clientes.

Versión 1: 4 clientes separados (ClienteSuma, ClienteResta, ClienteMultiplica, ClienteDivide) cada uno conectándose a su propio servidor de operación.

Versión 2: Un solo cliente que se conecta a 4 servidores independientes (ServicioSuma, ServicioResta, ServicioMultiplica, ServicioDivide). Esta versión es más limpia y reutilizable.

2. Arquitectura Orientada a Servicios (SOA)

Evolución del modelo cliente/servidor. Agrega un Bus (Enterprise Service Bus – ESB) que centraliza la comunicación. Los servicios son independientes y administrados centralizadamente. Usa estándares como XML, JSON, SOAP, REST.

Ventajas	Desventajas
Reutiliza sistemas existentes Bajo nivel de acoplamiento Escalabilidad, mantenibilidad, interoperabilidad	El BUS es punto único de falla Complejidad en entornos grandes

Ejemplo – Sistema Loto (SOA):

Servicios: Jugar (ingreso apuesta), Sortear (números ganadores), Ver (resultado de apuesta). Datos compartidos: Apuesta, Ganadores, Resultados. Todos los servicios se comunican a través del Bus ESB.

3. Modelo de Capas

Conjunto de capas que ofrecen servicios específicos con interfaces claramente definidas. La comunicación es solo entre capas vecinas. Permite desarrollo independiente de cada capa.

Ventajas	Desventajas
----------	-------------

Desarrollo incremental Flexible Mantenible	Difícil estructuración Dependencias cruzadas Baja performance
--	---

Ejemplo – Sistema Bancario (Capas):

Operaciones: Consulta saldo, Depósito, Giro, Transferencia. La transferencia depende del giro y del depósito; el giro depende de la consulta de saldo.

Capa UI (Presentación)	Consulta saldo UI · Giros UI · Depósitos UI · Transferencias UI
Capa GW (Gateway)	Consulta saldo GW1/GW2 · Giros GW · Depósitos GW · Transferencias GW
Capa BE (Business)	Consulta saldo BE · Giros BE · Depósitos BE · Transferencias BE
Capa DA (Data Access)	Giros DA · Depósitos DA

4. Modelo de Repositorio

Útil cuando muchas aplicaciones deben compartir grandes volúmenes de datos. Gestión centralizada (o distribuida) del almacenamiento. Existen dos variantes: modelo pasivo (las aplicaciones leen/escriben directamente) y modelo proactivo (el repositorio notifica cambios a las aplicaciones).

Ventajas	Desventajas
Modo eficiente de compartir datos Separa lógica de negocio del acceso a datos Admin. centralizada Seguridad, escalabilidad, mantenibilidad	Difícil cambio del modelo de datos Política centralizada de administración Posible punto único de falla

5. Objetos Distribuidos

Cada componente (objeto) define sus datos y métodos. Los objetos se comunican a través de un ORB (Object Request Broker). Arquitectura compleja pero muy flexible.

Ventajas	Desventajas
Diseño flexible Fácil agregar objetos Configuración dinámica Escalabilidad, mantenibilidad	Compleja construcción Bajo rendimiento

6. Arquitectura Cloud

Externalización de servicios computacionales bajo modelo de pago por uso (recursos elásticos). Tres niveles de servicio:

- **IaaS** (Infrastructure as a Service): infraestructura virtualizada (servidores, redes, almacenamiento).
- **PaaS** (Platform as a Service): plataforma de desarrollo y despliegue.
- **SaaS** (Software as a Service): aplicación completa entregada como servicio.

Ventajas	Desventajas
----------	-------------

Servicios ubicuos Reducción de costos Disponibilidad, escalabilidad, elasticidad Flexibilidad y movilidad	Dependencia de ente externo Riesgos de seguridad y confidencialidad Dependencia de conectividad
--	---

MÓDULO 4 · Arquitecturas Genéricas: Modelos de Control

El modelo de control define cómo se gestiona el flujo de ejecución entre componentes. Existen dos grandes enfoques:

Control Centralizado

Modelo Call-Return

Un componente controla la ejecución llamando a otros en secuencia. Cada llamada espera una respuesta antes de continuar.

- Simple y predecible
- Testeable con facilidad
- Rígido (poco adaptable)
- Bloqueante
- Complejo manejo de excepciones

Modelo Administrador (Manager)

Un componente central coordina la ejecución de otros procesos (útil en sistemas concurrentes).

- No bloqueante
- Coordinación de procesos paralelos
- Lógica centralizada en el administrador
- Posible cuello de botella

Control Basado en Eventos

Control descentralizado: los componentes responden a eventos generados externamente. No bloqueante por naturaleza. Dos variantes principales:

Transmisión Múltiple (Broadcast / Publicador-Suscriptor)

Los componentes publican servicios, gatillan eventos y suscriben a eventos de otros. La activación es descentralizada.

Ventajas	Desventajas
Activación descentralizada Distribución libre de componentes Evolución simple del sistema	Respuesta incierta (no se garantiza orden) Varios manejadores pueden responder al mismo evento

Manejo de Interrupciones

Usado en sistemas en tiempo real. Cada tipo de interrupción tiene su propio manejador que responde inmediatamente, sin bloqueo, y con capacidad de procesamiento paralelo.

CUADRO COMPARATIVO: Arquitecturas Genéricas

Arquitectura	Idea Central	Mejor para...	Cuidado con...
Cliente/Servidor	Clientes consumen servicios de servidores especializados	Sistemas distribuidos simples, APIs web	Performance y datos no compartidos
SOA	Servicios independientes comunicados por un Bus ESB	Integración empresarial, reuso de sistemas	El Bus como punto único de falla
Capas	Procesamiento por capas con interfaces definidas	Aplicaciones empresariales como sistemas bancarios	Dependencias cruzadas y baja performance
Repositorio	Datos compartidos centralmente	Sistemas con múltiples apps sobre los mismos datos	Cambios en el modelo de datos y punto único de falla
Objetos Distribuidos	Objetos se comunican vía ORB	Sistemas muy flexibles y dinámicos	Complejidad y bajo rendimiento
Cloud	Servicios externos elásticos (IaaS/PaaS/SaaS)	Startups, apps con carga variable, movilidad	Dependencia externa y seguridad/confidencialidad

CUADRO COMPARATIVO: Modelos de Control

Modelo	Mecanismo	Ventajas clave	Desventajas clave
Call-Return (Centralizado)	Un componente llama a otros secuencialmente	Simple, predecible, testeable	Rígido, bloqueante
Administrador (Centralizado)	Un manager coordina procesos concurrentes	No bloqueante, coordinado	Cuello de botella posible
Broadcast (Eventos)	Publicador-suscriptor, eventos distribuidos	Flexible, evolutivo, descentralizado	Respuesta incierta, múltiples manejadores
Interrupciones (Eventos)	Manejadores por tipo de interrupción	Tiempo real, inmediato, paralelo	Complejidad de manejadores

MÓDULO 5 · Ejercicios Resueltos (Ayudantía)

Ejercicio 1 – Calculadora (Cliente/Servidor)

Enunciado: Una empresa quiere ofrecer a sus clientes una calculadora con las 4 operaciones básicas: Sumar, Restar, Multiplicar, Dividir.

Versión 1 – Un cliente por operación:

Se crean 4 clientes independientes (ClienteSuma, ClienteResta, ClienteMultiplica, ClienteDivide), cada uno conectado a su propio servidor de operación. Problema: duplicación de clientes, difícil de mantener.

Versión 2 – Un cliente, múltiples servidores (recomendada):

Un solo cliente interactúa con 4 servidores independientes. El cliente envía la operación y los operandos al servidor correspondiente y recibe el resultado. Más limpio, escalable y mantenible.

Ejercicio 2 – Sistema Loto (SOA)

Enunciado: Sistema para el juego del Loto (6 números del 1 al 41). Debe permitir: ingresar apuestas, ingresar números ganadores, ver el premio obtenido.

Solución con SOA:

Servicio	Función	Datos que usa
Jugar	Ingresar apuesta del jugador	Apuesta
Sortear	Ingresar los números ganadores	Ganadores
Ver	Consultar el resultado de una apuesta	Resultados

Todos los servicios se comunican a través del Bus ESB, que centraliza el registro de servicios, el enrutamiento de mensajes, la seguridad y la escalabilidad.

Ejercicio 3 – Sistema Bancario (Capas)

Enunciado: Un banco necesita un sistema para manejar cuentas corrientes con: Consulta de saldo, Depósito, Giro y Transferencia entre cuentas.

Dependencias entre operaciones:

- Transferencia depende de → Giro + Depósito
- Giro depende de → Consulta de saldo

Estructura en capas:

Capa	Componentes y responsabilidades
UI (Presentación)	Interfaz de usuario para cada operación: Consulta UI · Giros UI · Depósitos UI · Transferencias UI
GW (Gateway)	Punto de entrada/salida hacia la lógica de negocio: Consulta GW1, GW2 · Giros GW · Depósitos GW · Transferencias GW
BE (Business)	Lógica del negocio: Consulta BE · Giros BE · Depósitos BE · Transferencias BE

DA (Data Access)	Acceso a la base de datos: Giros DA · Depósitos DA
-----------------------------------	---